In [96]:

```
data={
    'Customer':['A','B','C','D','E'],

    'Annual Income':[20,22,80,82,45],
    'Spending Score':[25,30,85,90,50]

}
```

In [143…

```
data
```

Out[143…

```
{'Customer': ['A', 'B', 'C', 'D', 'E'],
 'Annual Income': [20, 22, 80, 82, 45],
 'Spending Score': [25, 30, 85, 90, 50]}
```

In [145…

```
import pandas as pd
df=pd.DataFrame(data)
```

In [147…

```
df
```

Out[147…

	Customer	Annual Income	Spending Score
0	A	20	25
1	B	22	30
2	C	80	85
3	D	82	90
4	E	45	50

In [149…

```
X=df[['Annual Income','Spending Score']]
```

In [151…

```
X
```

Out[151…

	Annual Income	Spending Score
0	20	25
1	22	30
2	80	85
3	82	90
4	45	50

In [153…

```
from sklearn.cluster import KMeans

km=KMeans(
    n_clusters=2,
    random_state=42
)
```

In [155…

```
model=km.fit(X)
```

```
C:\Users\Lenovo\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available t
hreads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```

In [157…

```
model.labels_
```

Out[157…

```
array([0, 0, 1, 1, 0])
```

In [159…

```
#df['cluster']=model.labels_
```

In [161…

```
df
```

Out[161…

	Customer	Annual Income	Spending Score
0	A	20	25
1	B	22	30
2	C	80	85
3	D	82	90
4	E	45	50

In [163…

```
model.cluster_centers_
```

Out[163…

```
array([[29. , 35. ],
       [81. , 87.5]])
```

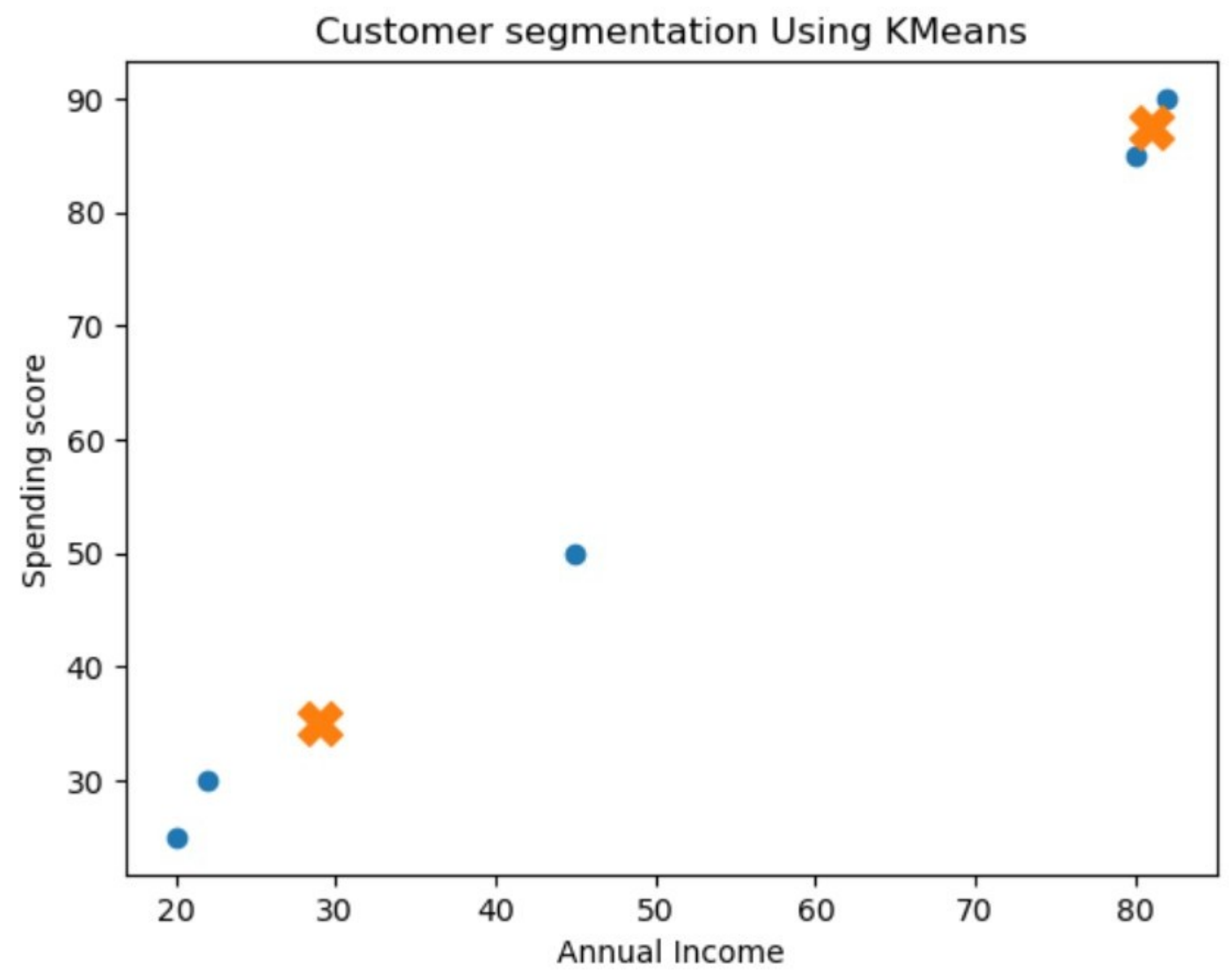
In [168…

```
import matplotlib.pyplot as plt

plt.scatter(
    X.iloc[:,0],
    X.iloc[:,1],
    #c=df['cluster']
)

plt.scatter(
    model.cluster_centers_[:,0],
    model.cluster_centers_[:,1],
    marker='X',
    s=200
)

plt.xlabel("Annual Income")
plt.ylabel("Spending score")
plt.title("Customer segmentation Using KMeans")
plt.show()
```



Elbow Method

In [ ]:

```
WCSS/Inertia
```

In [170…

```
wcss=[]

for k in range(1,5):
    km=KMeans(
        n_clusters=k,
        random_state=42
    )
    km.fit(X)
    wcss.append(km.inertia_)
```

```
C:\Users\Lenovo\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available t
hreads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\Lenovo\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available t
hreads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\Lenovo\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available t
hreads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\Lenovo\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available t
hreads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```

In [172…

```
wcss
```

Out[172…

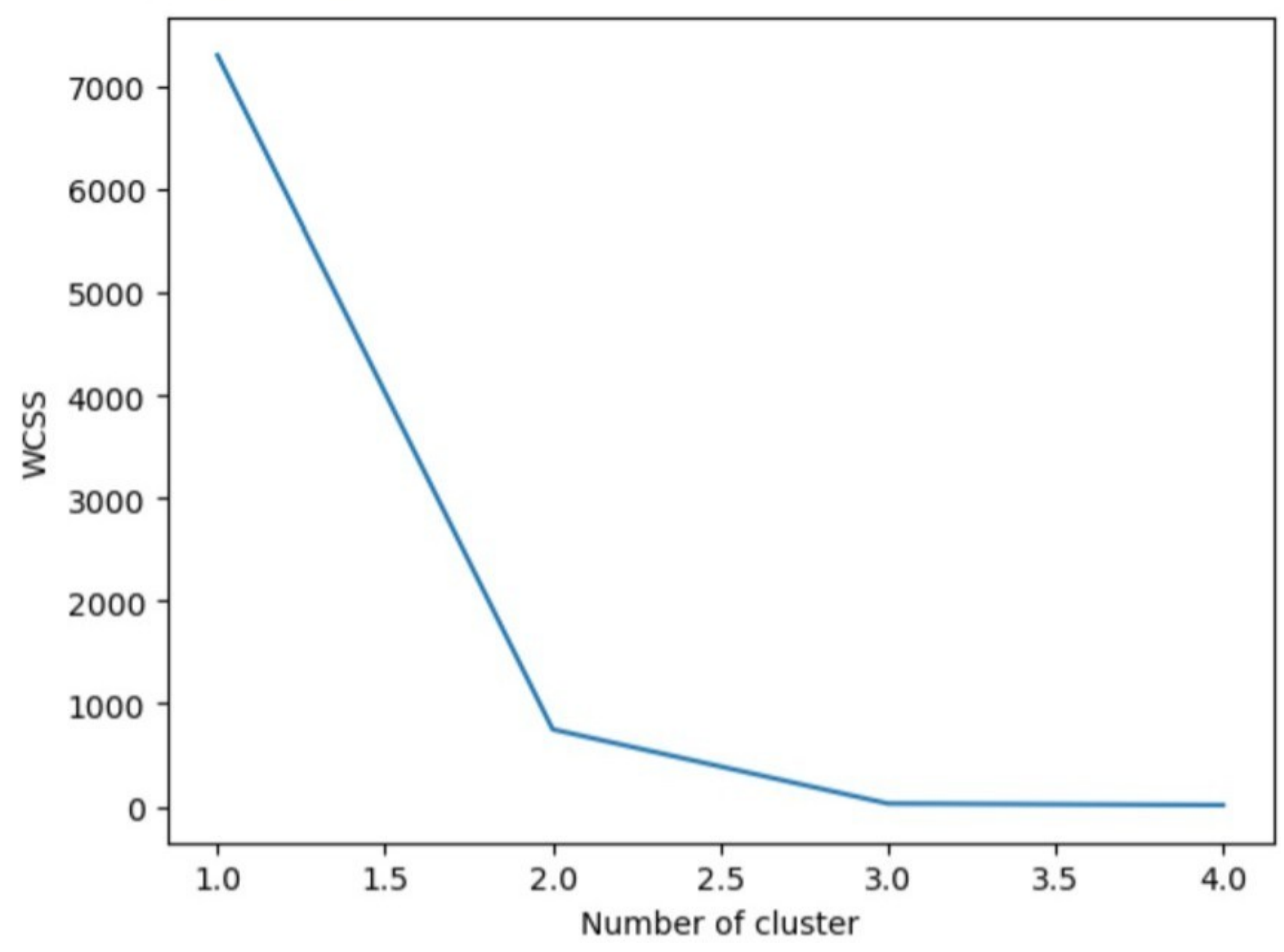
```
[7302.8, 750.5, 29.0, 14.5]
```

In [176…

```
plt.plot(range(1,5),wcss)
plt.xlabel('Number of cluster')
plt.ylabel('WCSS')
```

Out[176…

```
Text(0, 0.5, 'WCSS')
```



In [178…

```
km.inertia_
```

Out[178…

```
14.5
```

In [ ]:

```
wcss --->It means how close the data points are to their cluster centroid

Wcss --->High value
WCSS --->low value
```

In [ ]: